

TEORÍA (8 puntos)

Apellidos y nombre:

Pregunta 1. Enumera tres técnicas de depuración por fuerza bruta

Pregunta 2. ¿Qué significan las palabras reservadas `requires` y `ensures` de JML?

Pregunta 3. Indica el significado de las siguientes etiquetas para métodos utilizadas en JUnit 4:

`@Test`:

`@Before`:

`@After`:

`@BeforeClass`:

`@AfterClass`:

Pregunta 4. ¿En qué consiste la base de las pruebas (*test basis*)?:

Pregunta 5. Dada una estimación previa del esfuerzo en nº de horas requerido por un proceso de pruebas, describe brevemente dos técnicas que permitan repartir ese número de horas entre distintas actividades o componentes:

Pregunta 6. ¿En qué consiste un *script* de pruebas?

Pregunta 7. Enumera tres métricas que permitan informar acerca de la productividad, o el progreso y la calidad del proceso de pruebas:

Pregunta 8. ¿Por qué es relevante la Interfaz de Usuario en las pruebas de usabilidad?

Pregunta 9. Relaciona las siguientes herramientas de prueba de GUI con el tipo de aplicación o pruebas que facilitan:

Sikuli	1●	●a	Prueba de aplicaciones Java
Exerciser Monkey	2●	●b	Prueba de aplicaciones Web
Maveryx	3●	●c	Pruebas de aceptación
Selenium	4●	●d	Pruebas basadas en capturas de pantalla
Cucumber	5●	●e	Pruebas de estrés

Pregunta 10. Enumera al menos tres características que se deberían comprobar durante la evaluación estática del documento correspondiente a un catálogo de requisitos.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

Dado el siguiente código parcial de la clase `ListaOrdenada`, diseñad pruebas para el método `insertar()` utilizando la técnica de pruebas de caminos con nivel de profundidad igual a 3:

1. Obtened primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/condiciones y nodos que se indica en el propio código (0,25 puntos).
2. Aplicad la técnica de pruebas de caminos con profundidad 3 para obtener el conjunto de caminos completos correspondiente (0,75 puntos).
3. Finalmente, estableced casos de prueba para los caminos obtenidos, incluyendo el parámetro explícito del método, su valor devuelto y el estado inicial y final de los atributos del objeto `ListaOrdenada` sobre el que se invocará al método `insertar()` (1 punto).

```
/**
 * Los objetos de la clase «ListaOrdenada» implementan una lista de enteros
 * de como mucho «CAPACIDAD_MAXIMA» elementos, ordenados siempre de menor a
 * mayor.
 */
public class ListaOrdenada {
    public static final int CAPACIDAD_MAXIMA = 5;

    private int[] datos;
    private int numDatos;

    /**
     * Crea una ListaOrdenada vacía (sin elementos).
     */
    public ListaOrdenada() {
        datos = new int[CAPACIDAD_MAXIMA];
        numDatos = 0;
    }

    /**
     * Si no se ha alcanzado la capacidad máxima de la lista, añade el
     * elemento dado a esta lista, manteniendo sus valores ordenados de
     * menor a mayor. En caso contrario, no se modifica la lista.
     *
     * @param elemento el elemento a añadir a la lista
     * @returns el índice del elemento que se acaba de insertar en la lista
     * o -1 si el elemento no se ha podido insertar por tratarse originalmente
     * de una lista en la que se había alcanzado ya la capacidad máxima.
     */
    int insertar(final int elemento) {
        A if (numDatos < CAPACIDAD_MAXIMA) {
            B int i = this.numDatos - 1;
            while ( C i >= 0
                    D && this.datos[i] > elemento) {
                E this.datos[i + 1] = this.datos[i];
                i--;
            }
            // i < 0 || lista.datos[i] <= elemento
            F this.datos[i + 1] = elemento;
            this.numDatos++;
            return i + 1;
        }
        else {
            G return -1;
        }
    }
    ...
}
```